TASK 3

Flicker-Free LCD Display Engine

Real-Time EV Battery Management System (BMS)

Wokwi ESP32 Simulation — Final Demonstration Report

Evidence source: only the latest uploaded Wokwi demonstration video was used for the screenshots in this report.

Final timing: LCD refresh = 250 ms; automatic page rotation = 5 seconds; startup screen = 2 seconds.

Submission contents

This report documents the circuit, display architecture, non-blocking timing, page rotation, fault override behavior, test results, and the final Arduino sketch.

1. Task Requirement

Create an optimized LCD rendering engine that updates only portions of the display whose values have changed, eliminating unnecessary screen clearing and visible flickering. The display must automatically rotate among at least three information pages using a non-blocking timer. During a critical fault, the LCD must immediately override the normal display and show a dedicated fault screen until the fault is explicitly cleared. The refresh interval must be justified using LCD characteristics and information importance.

2. Implemented Solution

The implementation uses a 16×2 I2C LCD at address 0x27. The normal interface contains three pages: Battery, System, and Telemetry. Page changes use millis()-based timing rather than delay(). LCD rendering keeps oldLine1 and oldLine2 caches and rewrites a line only when its content changes. A latched fault state has priority over the normal page state.

2.1 Normal Pages

Page	LCD content	Purpose
Battery	SOC, pack voltage, minimum and maximum cell voltage	Battery condition
System	SYSTEM: NORMAL / RELAY: ON	System state
Telemetry	Current, imbalance, weakest/strongest cell	Live telemetry

2.2 Critical Fault Screens

Blue button → FROZEN SENSOR; Yellow button → VOLTAGE JUMP; Red button → CELL RANGE. When a fault is latched, the normal page rotation is suspended and the LCD immediately displays the dedicated critical screen. The relay is turned off, the green LED is turned off, the red LED is turned on, and the buzzer is turned on.

3. Hardware and Pin Mapping

Component	GPIO / interface	Function
Cell 1 potentiometer	GPIO32	Cell 1 voltage simulation
Cell 2 potentiometer	GPIO33	Cell 2 voltage simulation
Cell 3 potentiometer	GPIO34	Cell 3 voltage simulation
Cell 4 potentiometer	GPIO35	Cell 4 voltage simulation
SOC potentiometer	GPIO36	State of charge simulation
Current potentiometer	GPIO39	Discharge/current simulation
Relay	GPIO23	Safety isolation / load control
Green LED	GPIO25	Normal indication
Red LED	GPIO27	Fault indication
Buzzer	GPIO14	Fault alarm
Blue button	GPIO16	Frozen fault
Yellow button	GPIO17	Jump fault
Red button	GPIO18	Range fault
LCD SDA	GPIO21	I2C data
LCD SCL	GPIO22	I2C clock
LCD	0x27, 16×2	Display

4. Software Architecture

The main loop follows a non-blocking sequence: read sensors → read buttons → update fault state → update safety outputs → rotate page when the 5-second timer expires → refresh the LCD every 250 ms → process serial commands. The LCD engine compares each new line against its cached previous value before writing.

The fault state is given higher priority than page selection. Therefore, once faultLatched becomes true, updateLCD() renders the critical screen regardless of which normal page was active.

5. Timing and Flicker-Free Rendering

Parameter	Value	Reason
LCD refresh interval	250 ms (4 Hz)	Fast enough for BMS information while limiting unnecessary I2C/LCD writ
Page interval	5 seconds	Provides enough reading time on a 16×2 character LCD.
Startup display	2 seconds	Allows the project identification screen to be seen before live data.

The implementation does not continuously call lcd.clear() during normal operation. Instead, it clears only the text line being replaced by writing 16 spaces and then writing the new line. More importantly, the write is skipped completely when the generated line is identical to the cached line. This reduces repeated display writes and avoids the visible flicker caused by repeatedly clearing and redrawing the LCD.

The page timer uses unsigned long subtraction (now - lastPageChange), which remains non-blocking and is suitable for millis()-based embedded timing. Critical fault handling is not delayed by the 5-second page timer.

6. Final Demonstration Screenshots

All screenshots in this section were extracted only from the latest uploaded Wokwi video. No screenshots from the earlier uploaded videos were used.

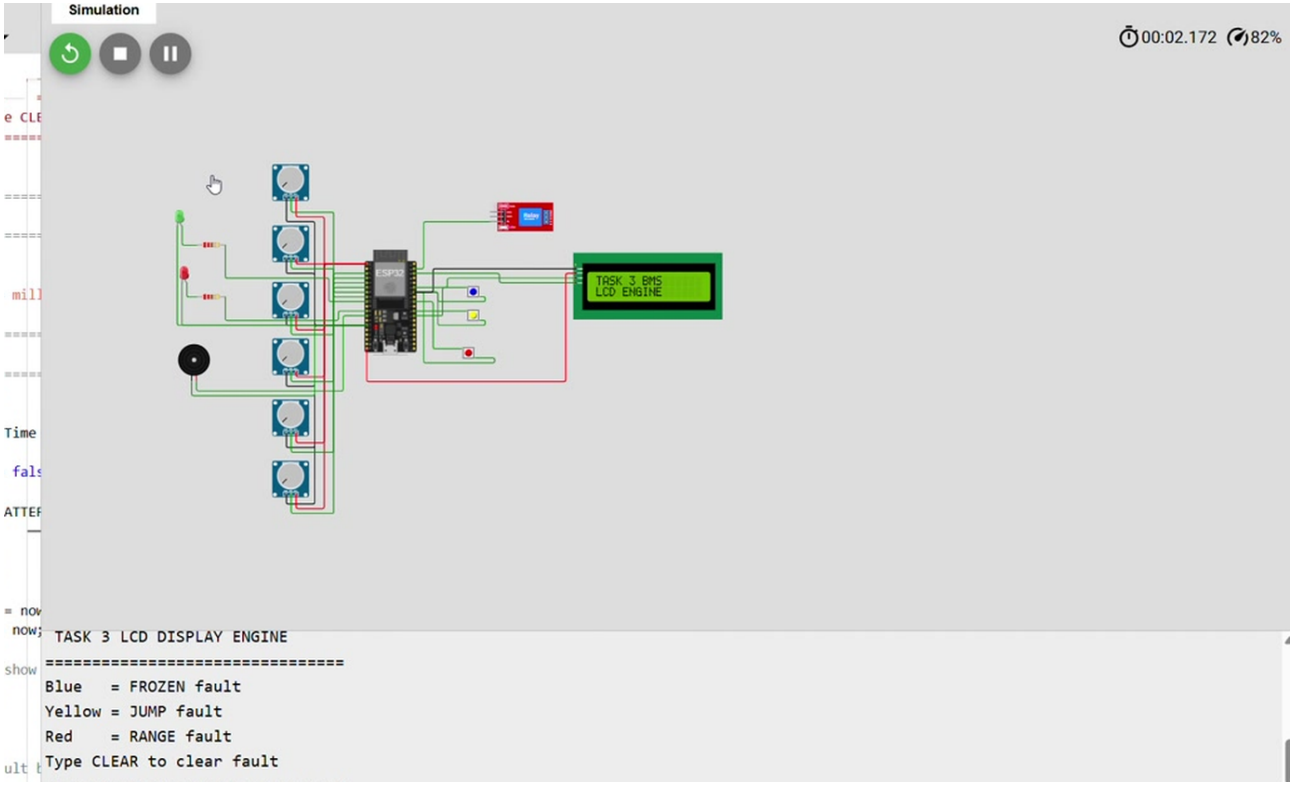


Figure 1. Startup screen — TASK 3 BMS / LCD ENGINE.

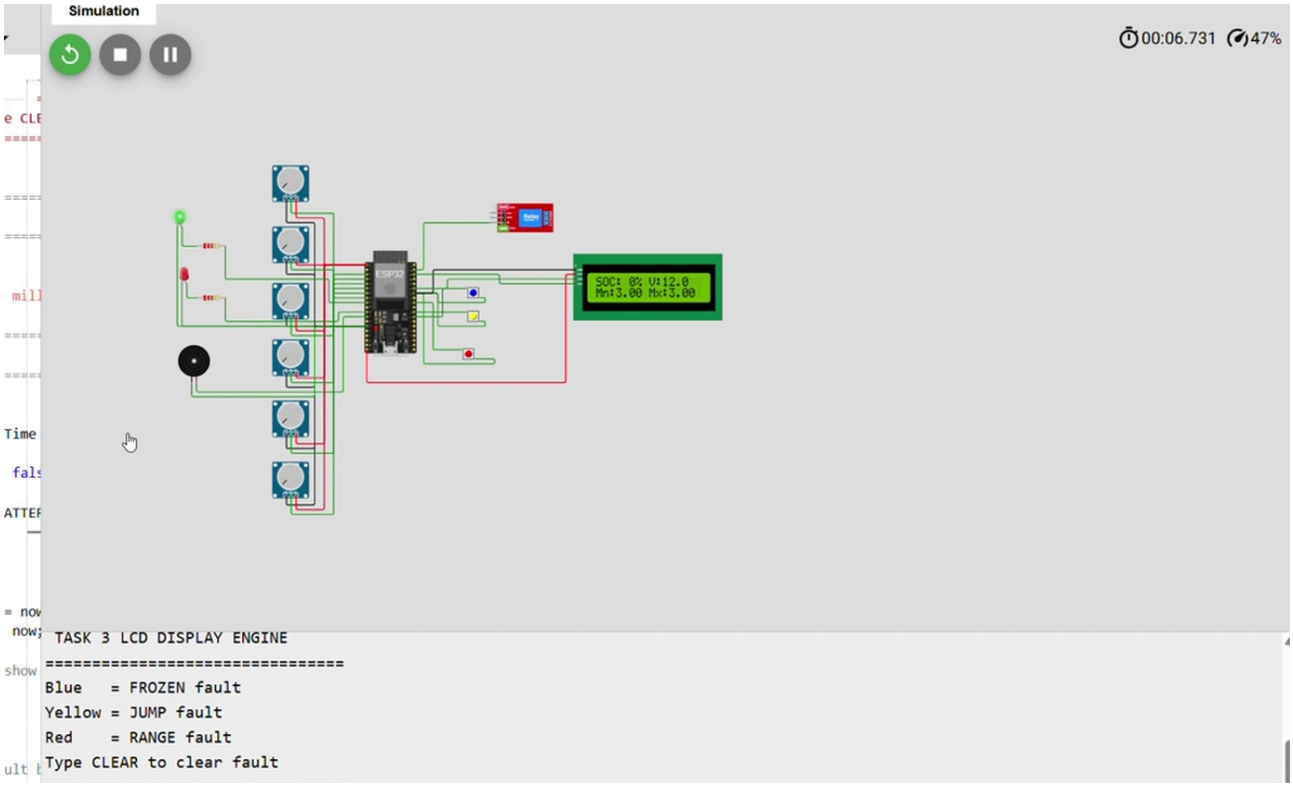


Figure 2. Battery page — SOC, pack voltage, minimum and maximum cell voltage.

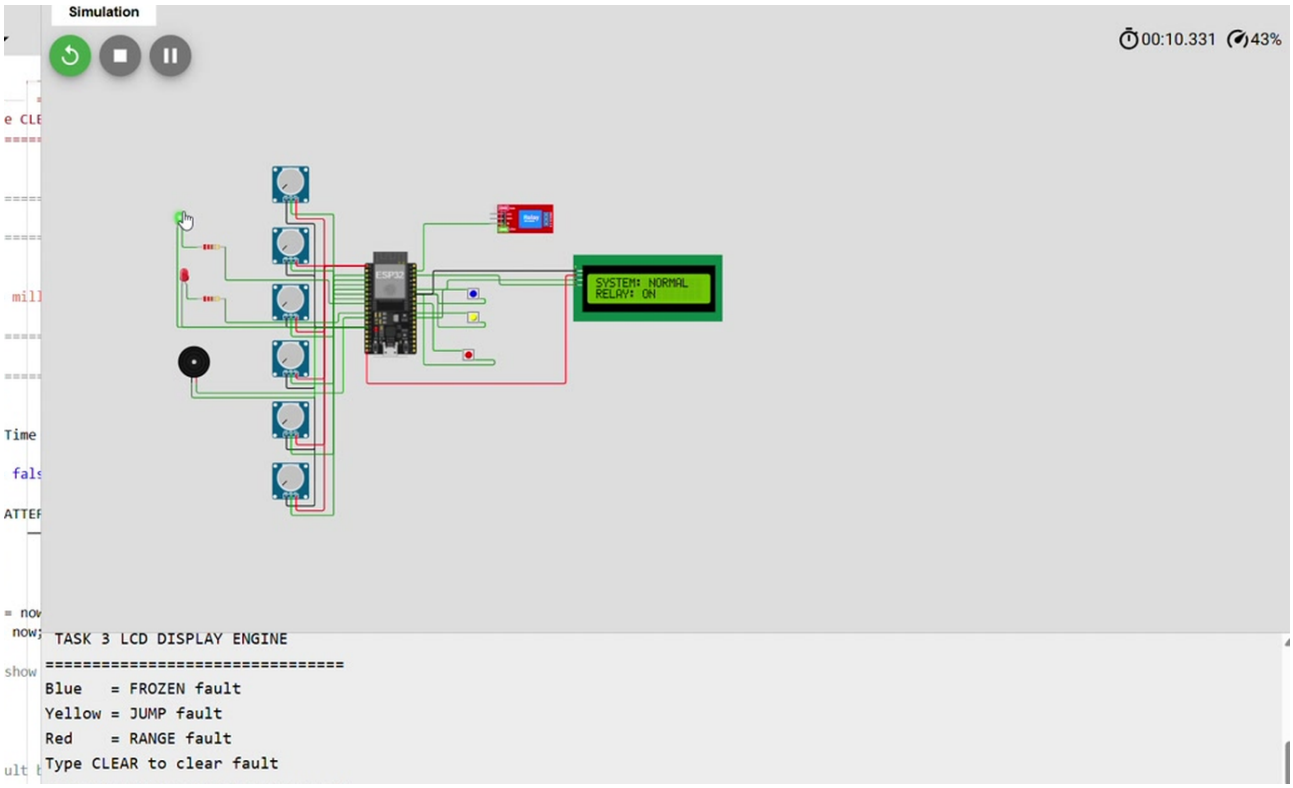


Figure 3. System page — SYSTEM: NORMAL / RELAY: ON.

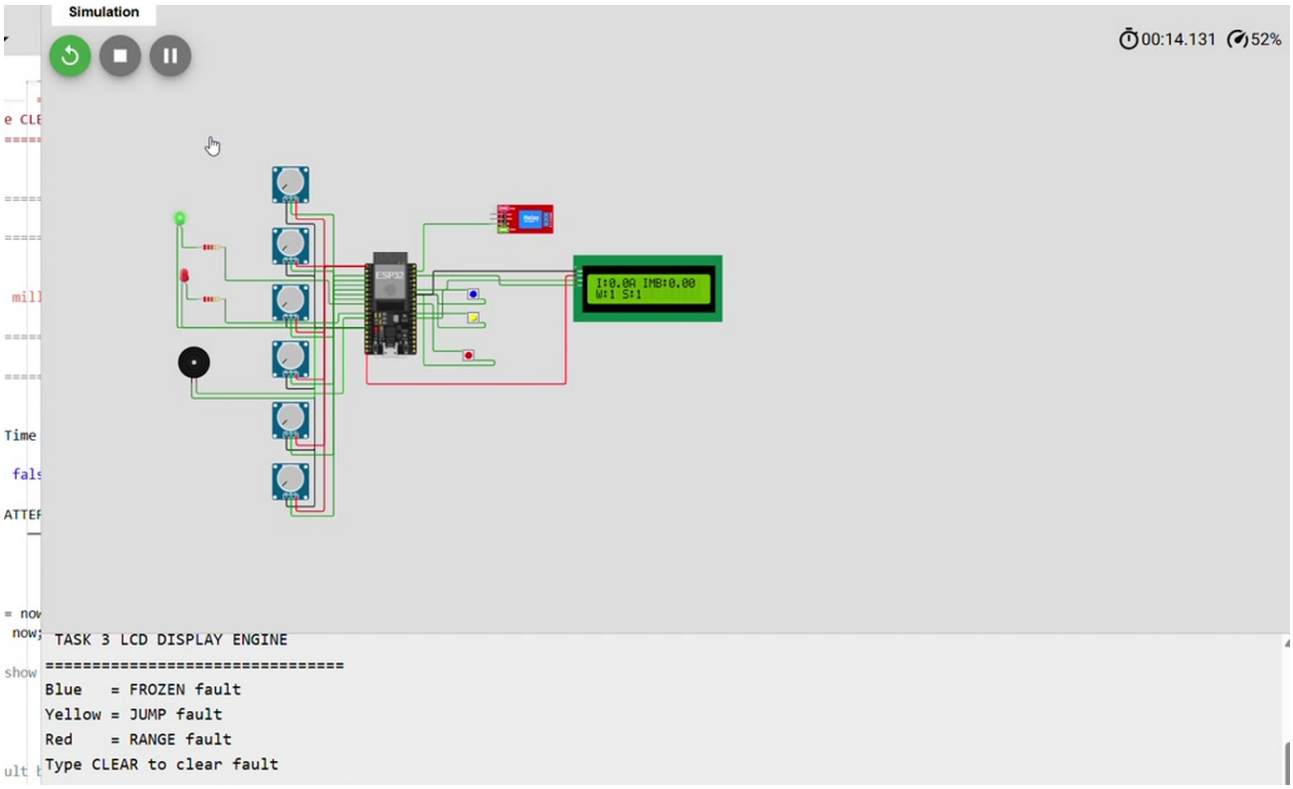


Figure 4. Telemetry page — current, imbalance, weakest and strongest cells.

7. Fault and Recovery Evidence

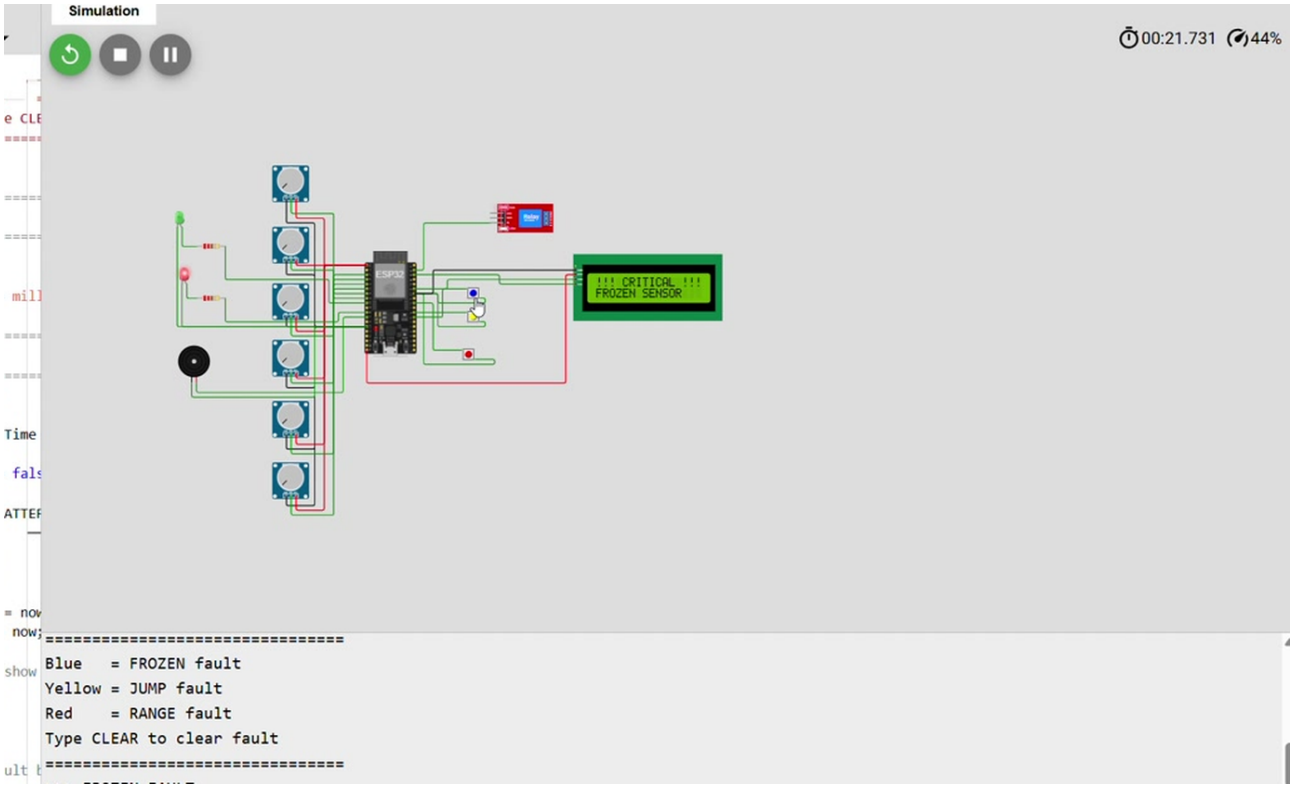


Figure 5. Frozen fault — LCD overrides the normal page and displays FROZEN SENSOR.

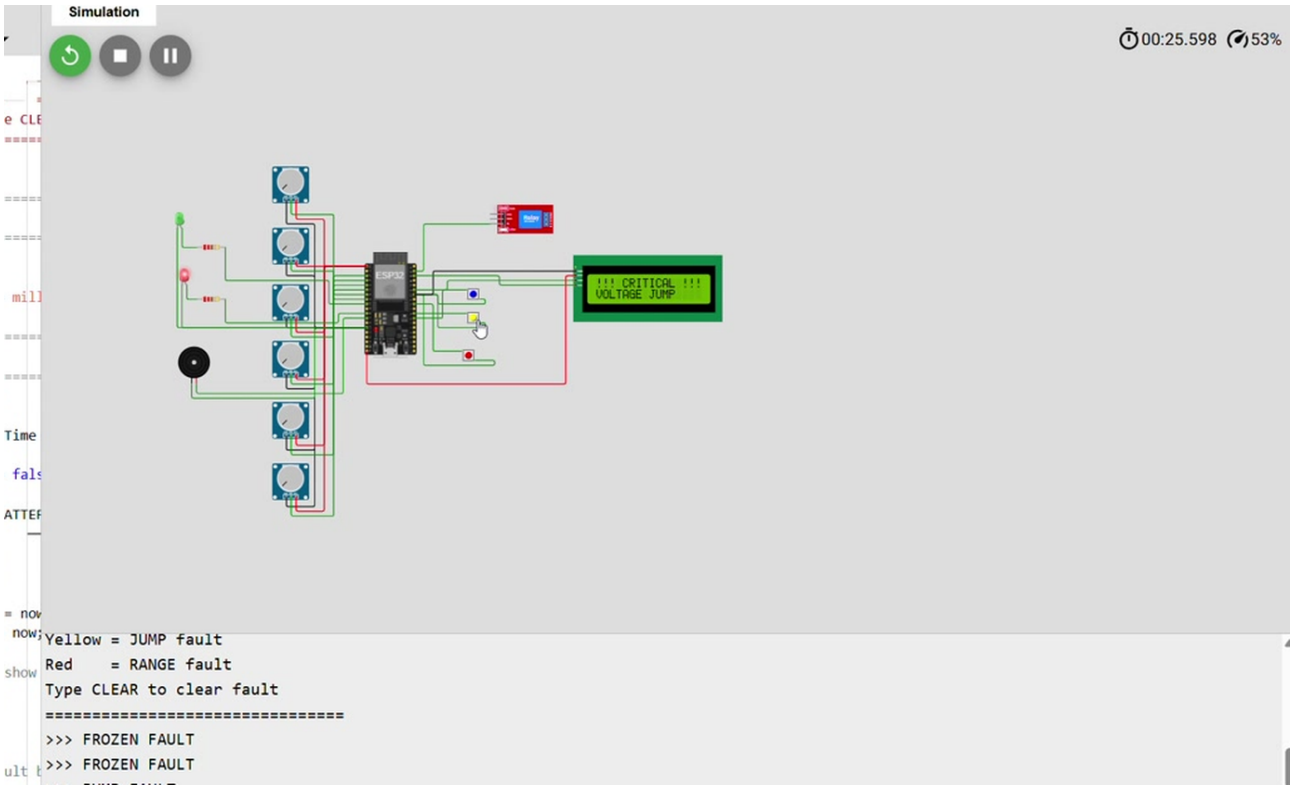


Figure 6. Jump fault — LCD displays VOLTAGE JUMP.

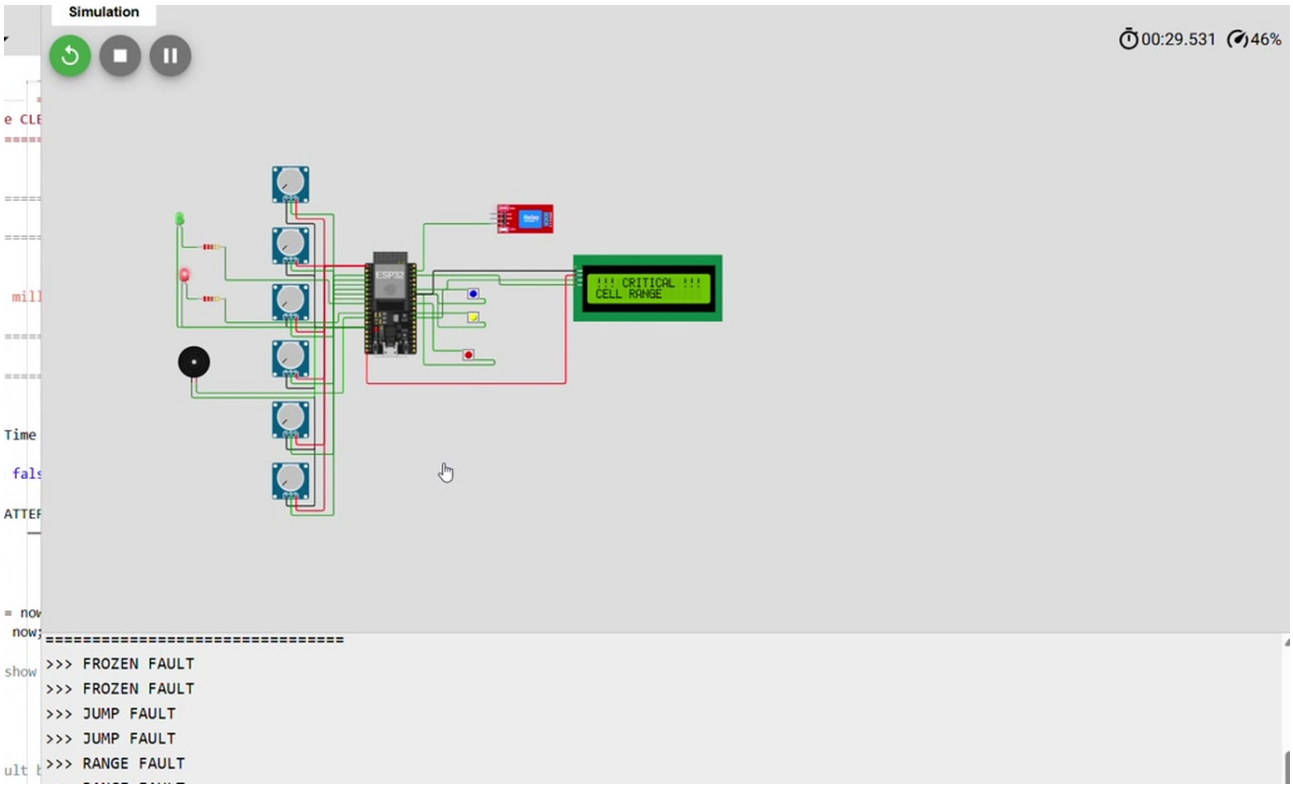


Figure 7. Range fault — LCD displays CELL RANGE.

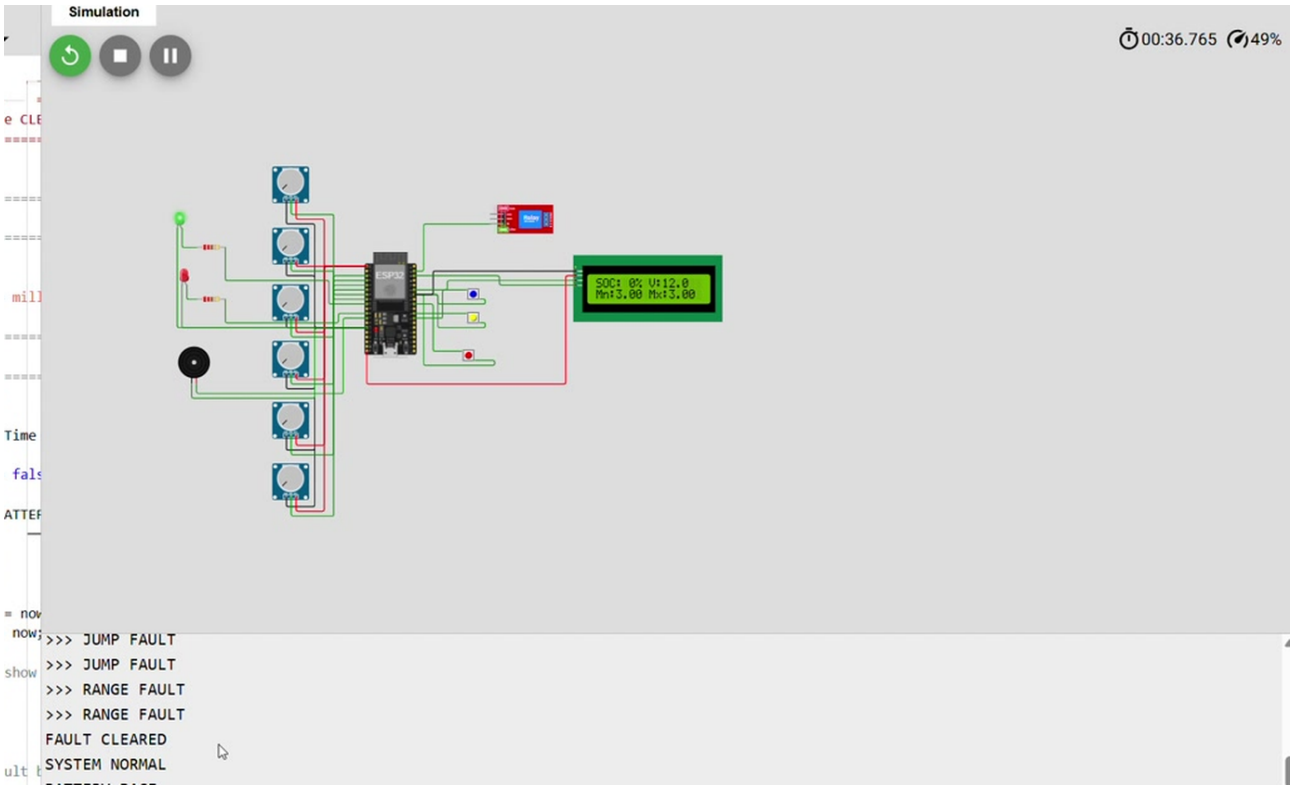


Figure 8. Fault cleared — Serial Monitor shows FAULT CLEARED / SYSTEM NORMAL and the LCD returns to the Battery page.

8. Test Procedure and Results

Test	Procedure	Expected result	Result
1	Run simulation and observe page rotation.	Battery → System → Telemetry → Battery.	PASS
2	Press blue Frozen button.	Critical screen: FROZEN SENSOR; safety outputs change.	PASS
3	Clear fault, then press yellow Jump button.	Critical screen: VOLTAGE JUMP.	PASS
4	Clear fault, then press red Range button.	Critical screen: CELL RANGE.	PASS
5	Enter CLEAR in Serial Monitor.	Fault clears and Battery page resumes.	PASS
6	Observe normal LCD updates.	No continuous lcd.clear(); changed content is rewritten.	PASS

Observed evidence in the final video includes the normal display pages, FROZEN FAULT, JUMP FAULT, RANGE FAULT, and the final FAULT CLEARED / SYSTEM NORMAL / BATTERY PAGE sequence.

9. Result

Task 3 is demonstrated as a working Wokwi implementation. The LCD rotates through three information pages with a non-blocking timer, updates only changed display content, and gives critical faults priority over the normal interface. Faults remain latched until the explicit CLEAR command is received.

10. Conclusion

The final design satisfies the core Task 3 requirements for a flicker-free LCD display engine: partial/conditional rendering, three automatic information pages, non-blocking timing, immediate fault override, and explicit fault recovery.

Appendix A — Final sketch.ino

The complete final Arduino sketch is also supplied separately as Task3_Final_sketch.ino.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

// Cell inputs
const int CELL1_PIN = 32;
const int CELL2_PIN = 33;
const int CELL3_PIN = 34;
const int CELL4_PIN = 35;

// SOC and current
const int SOC_PIN = 36;
const int CURRENT_PIN = 39;

// Safety outputs
const int RELAY_PIN = 23;
const int GREEN_LED = 25;
const int RED_LED = 27;
const int BUZZER = 14;

// Fault buttons
const int FROZEN_BUTTON = 16;
const int JUMP_BUTTON   = 17;
const int RANGE_BUTTON  = 18;

// Timing
const unsigned long LCD_REFRESH_INTERVAL = 250;
const unsigned long PAGE_INTERVAL = 5000;
const unsigned long STARTUP_TIME = 2000;

enum Page {
  BATTERY_PAGE,
  SYSTEM_PAGE,
  TELEMETRY_PAGE
};

Page currentPage = BATTERY_PAGE;

enum FaultType {
  NO_FAULT,
  FAULT_FROZEN,
  FAULT_JUMP,
  FAULT_RANGE
};

FaultType faultType = NO_FAULT;
bool faultLatched = false;

bool startupScreen = true;
unsigned long startupTime = 0;
unsigned long lastLCDUpdate = 0;
unsigned long lastPageChange = 0;

float cellVoltage[4];
float packVoltage = 0.0;
float minVoltage = 0.0;
float maxVoltage = 0.0;
float imbalance = 0.0;
int weakestCell = 1;
int strongestCell = 1;
float soc = 0.0;
float current = 0.0;

String oldLine1 = "";
String oldLine2 = "";

void readSensors();
void readButtons();
void checkFaults();
void updateOutputs();
void updateLCD();
```

```
void readSerial();
void forceLCDRefresh();

void setup()
{
  Serial.begin(115200);

  pinMode(RELAY_PIN, OUTPUT);
  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(BUZZER, OUTPUT);

  pinMode(FROZEN_BUTTON, INPUT_PULLUP);
  pinMode(JUMP_BUTTON, INPUT_PULLUP);
  pinMode(RANGE_BUTTON, INPUT_PULLUP);

  lcd.init();
  lcd.backlight();

  digitalWrite(RELAY_PIN, LOW);
  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, LOW);
  digitalWrite(BUZZER, LOW);

  lcd.setCursor(0, 0);
  lcd.print("TASK 3 BMS      ");
  lcd.setCursor(0, 1);
  lcd.print("LCD ENGINE      ");

  startupScreen = true;
  startupTime = millis();

  currentPage = BATTERY_PAGE;
  lastPageChange = millis();
  lastLCDUpdate = millis();

  Serial.println();
  Serial.println("================================");
  Serial.println(" TASK 3 LCD DISPLAY ENGINE");
  Serial.println("================================");
  Serial.println("Blue   = FROZEN fault");
  Serial.println("Yellow = JUMP fault");
  Serial.println("Red    = RANGE fault");
  Serial.println("Type CLEAR to clear fault");
  Serial.println("================================");
}

void loop()
{
  unsigned long now = millis();

  if (startupScreen)
  {
    if (now - startupTime >= STARTUP_TIME)
    {
      startupScreen = false;
      currentPage = BATTERY_PAGE;
      oldLine1 = "";
      oldLine2 = "";
      lastPageChange = now;
      lastLCDUpdate = now;

      readSensors();
      updateLCD();
    }

    readButtons();
    return;
  }

  readSensors();
  readButtons();
```

```
  checkFaults();
  updateOutputs();

  if (!faultLatched)
  {
    if (now - lastPageChange >= PAGE_INTERVAL)
    {
      lastPageChange = now;

      if (currentPage == BATTERY_PAGE)
        currentPage = SYSTEM_PAGE;
      else if (currentPage == SYSTEM_PAGE)
        currentPage = TELEMETRY_PAGE;
      else
        currentPage = BATTERY_PAGE;

      oldLine1 = "";
      oldLine2 = "";
      updateLCD();
    }
  }

  if (now - lastLCDUpdate >= LCD_REFRESH_INTERVAL)
  {
    lastLCDUpdate = now;
    updateLCD();
  }

  readSerial();
}

void readSensors()
{
  int raw1 = analogRead(CELL1_PIN);
  int raw2 = analogRead(CELL2_PIN);
  int raw3 = analogRead(CELL3_PIN);
  int raw4 = analogRead(CELL4_PIN);

  cellVoltage[0] = 3.0 + ((float)raw1 / 4095.0) * 1.2;
  cellVoltage[1] = 3.0 + ((float)raw2 / 4095.0) * 1.2;
  cellVoltage[2] = 3.0 + ((float)raw3 / 4095.0) * 1.2;
  cellVoltage[3] = 3.0 + ((float)raw4 / 4095.0) * 1.2;

  int rawSOC = analogRead(SOC_PIN);
  soc = ((float)rawSOC / 4095.0) * 100.0;

  int rawCurrent = analogRead(CURRENT_PIN);
  current = ((float)rawCurrent / 4095.0) * 10.0;

  packVoltage =
    cellVoltage[0] + cellVoltage[1] +
    cellVoltage[2] + cellVoltage[3];

  minVoltage = cellVoltage[0];
  maxVoltage = cellVoltage[0];
  weakestCell = 1;
  strongestCell = 1;

  for (int i = 1; i < 4; i++)
  {
    if (cellVoltage[i] < minVoltage)
    {
      minVoltage = cellVoltage[i];
      weakestCell = i + 1;
    }

    if (cellVoltage[i] > maxVoltage)
    {
      maxVoltage = cellVoltage[i];
      strongestCell = i + 1;
    }
  }
```

```
  imbalance = maxVoltage - minVoltage;
}

void readButtons()
{
  static bool oldFrozen = HIGH;
  static bool oldJump = HIGH;
  static bool oldRange = HIGH;

  bool frozenNow = digitalRead(FROZEN_BUTTON);
  bool jumpNow   = digitalRead(JUMP_BUTTON);
  bool rangeNow  = digitalRead(RANGE_BUTTON);

  if (oldFrozen == HIGH && frozenNow == LOW)
  {
    faultLatched = true;
    faultType = FAULT_FROZEN;
    Serial.println(">>> FROZEN FAULT");
    oldLine1 = "";
    oldLine2 = "";
    updateOutputs();
    updateLCD();
  }

  if (oldJump == HIGH && jumpNow == LOW)
  {
    faultLatched = true;
    faultType = FAULT_JUMP;
    Serial.println(">>> JUMP FAULT");
    oldLine1 = "";
    oldLine2 = "";
    updateOutputs();
    updateLCD();
  }

  if (oldRange == HIGH && rangeNow == LOW)
  {
    faultLatched = true;
    faultType = FAULT_RANGE;
    Serial.println(">>> RANGE FAULT");
    oldLine1 = "";
    oldLine2 = "";
    updateOutputs();
    updateLCD();
  }

  oldFrozen = frozenNow;
  oldJump = jumpNow;
  oldRange = rangeNow;
}

void checkFaults()
{
  // The three Wokwi buttons provide explicit Task 3 fault tests.
  // Normal potentiometer ranges are intentionally limited to
  // approximately 3.0-4.2 V per cell.
}

void updateOutputs()
{
  if (faultLatched)
  {
    digitalWrite(RELAY_PIN, LOW);
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, HIGH);
    digitalWrite(BUZZER, HIGH);
  }
  else
  {
    digitalWrite(RELAY_PIN, HIGH);
    digitalWrite(GREEN_LED, HIGH);
```

```
    digitalWrite(RED_LED, LOW);
    digitalWrite(BUZZER, LOW);
  }
}

void updateLCD()
{
  String line1 = "";
  String line2 = "";

  if (faultLatched)
  {
    line1 = "!!! CRITICAL !!!";

    if (faultType == FAULT_FROZEN)
      line2 = "FROZEN SENSOR";
    else if (faultType == FAULT_JUMP)
      line2 = "VOLTAGE JUMP";
    else if (faultType == FAULT_RANGE)
      line2 = "CELL RANGE";
  }
  else if (currentPage == BATTERY_PAGE)
  {
    line1 = "SOC:" + String(soc, 0) +
            "% V:" + String(packVoltage, 1);

    line2 = "Mn:" + String(minVoltage, 2) +
            " Mx:" + String(maxVoltage, 2);
  }
  else if (currentPage == SYSTEM_PAGE)
  {
    line1 = "SYSTEM: NORMAL";
    line2 = "RELAY: ON";
  }
  else if (currentPage == TELEMETRY_PAGE)
  {
    line1 = "I:" + String(current, 1) +
            "A IMB:" + String(imbalance, 2);

    line2 = "W:" + String(weakestCell) +
            " S:" + String(strongestCell);
  }

  if (line1 != oldLine1)
  {
    lcd.setCursor(0, 0);
    lcd.print("                ");
    lcd.setCursor(0, 0);
    lcd.print(line1);
    oldLine1 = line1;
  }

  if (line2 != oldLine2)
  {
    lcd.setCursor(0, 1);
    lcd.print("                ");
    lcd.setCursor(0, 1);
    lcd.print(line2);
    oldLine2 = line2;
  }
}

void forceLCDRefresh()
{
  oldLine1 = "";
  oldLine2 = "";
  updateLCD();
}

void readSerial()
{
  if (Serial.available())
```

```
  {
    String command = Serial.readStringUntil('\n');
    command.trim();
    command.toUpperCase();

    if (command == "CLEAR")
    {
      faultLatched = false;
      faultType = NO_FAULT;
      currentPage = BATTERY_PAGE;
      lastPageChange = millis();

      updateOutputs();

      oldLine1 = "";
      oldLine2 = "";
      updateLCD();

      Serial.println("FAULT CLEARED");
      Serial.println("SYSTEM NORMAL");
      Serial.println("BATTERY PAGE");
    }
    else if (command == "STATUS")
    {
      Serial.println();
      Serial.println("------ BMS STATUS ------");
      Serial.print("SOC: ");
      Serial.print(soc, 1);
      Serial.println("%");

      Serial.print("Pack Voltage: ");
      Serial.print(packVoltage, 2);
      Serial.println(" V");

      Serial.print("Min Voltage: ");
      Serial.print(minVoltage, 2);
      Serial.println(" V");

      Serial.print("Max Voltage: ");
      Serial.print(maxVoltage, 2);
      Serial.println(" V");

      Serial.print("Imbalance: ");
      Serial.print(imbalance, 2);
      Serial.println(" V");

      Serial.print("Weakest Cell: ");
      Serial.println(weakestCell);

      Serial.print("Strongest Cell: ");
      Serial.println(strongestCell);

      Serial.print("Current: ");
      Serial.print(current, 2);
      Serial.println(" A");

      Serial.println(faultLatched ? "FAULT: ACTIVE" : "FAULT: NONE");
      Serial.println("------------------------");
    }
  }
}
```